

Security Analysis Report - AI Interview Platform (Frontend)

Prepared for production readiness review

1. Dependency Vulnerabilities (Action Required)

- **Issue**: `esbuild` (via `vite@<=6.4.1`) has a **moderate severity** vulnerability (GHSA-67mh-4wv8-2f99) that allows arbitrary requests to the dev server and response reading.
- **Impact**: Potential data leakage or server-side request SSRF in development environments. Less likely in production (Vite dev server not used), but still a risk if dev server is exposed.
- **Fix**: Run `npm audit fix --force` to upgrade `vite` to `8.0.14` (includes patched `esbuild`).
- **Note**: This is a **breaking change**; thoroughly test the app after upgrading (especially Vite plugin usage, build output).
- **Alternative**: If breaking changes are unacceptable, monitor for a vite patch that addresses the issue without major version bump (check `vite@6.4.2+`).

**2. Environment & Configuration Risks

- | Finding | Risk | Recommendation |
|--|---|--|
| `VITE_API_KEY` is empty in `.env.local` | If the backend expects this key for request authentication, missing key may cause auth failures. If the key is meant to be secret, exposing it via `VITE_` prefix is unsafe (anyone can view it in browser source). | - Confirm with backend team whether the key is required and if it should be secret. |
| - If secret, do not expose it to the frontend | - move authentication to backend-only channels or use a short-lived token flow. | |
| - If public, ensure it is properly set in production env. | | |
| `.env.local` contains `VITE_API_URL=http://localhost:8000` | Hard-coded dev URL will be baked into the build if not overridden. In production, this would point to `localhost` (breaking the app). | - Use environment-specific `.env` files (`.env.production`, `.env.staging`) and ensure CI/CD injects the correct `VITE_API_URL` at build time. |
| - Verify that the production build uses the correct URL (e.g., `https://api.example.com`). | | |
| `secure: false` in Vite proxy (vite.config.js) | Disables SSL certificate validation for proxied requests. In development this is fine for self-signed certs, but if similar settings were accidentally used in production (e.g., via a custom server), it could lead to MITM risks. | - Ensure this file is only used by Vite dev server and not bundled into production builds (it isn't, as Vite excludes vite.config.js from the build). |
| - No action needed, but keep under review. | | |

**3. Client-Side Storage & Authentication

- **Tokens stored in `localStorage`** (`auth\_token` and `token`).
- **Risk**: Vulnerable to theft via XSS (Cross-Site Scripting). If an attacker can inject JavaScript, they can steal tokens and impersonate users.
- **Recommendation**:
 1. Evaluate migrating to **HttpOnly, Secure cookies** (set by the backend) for session tokens - this prevents JavaScript access.
 2. If localStorage must be kept, implement **strict Content Security Policy (CSP)** to mitigate XSS, and sanitize all user-input rendered in the DOM.
- **Logout only clears localStorage; no backend logout call**.
- **Risk**: If the backend uses server-side sessions or token blacklisting, the old token may remain valid until expiration.
- **Recommendation**: Call a `/api/auth/logout` endpoint (or similar) during logout to invalidate the token server-side.

**4. Data Exposure & Logging

- **Console.error with potentially sensitive data** in `submitAnswer` (line ~200):

```

```javascript
console.error("🔴 Backend 500 detail:", text);
```

- Risk: If the backend returns error messages containing PII, tokens, or internal details, they could be exposed to users via browser console (visible in devtools, and potentially logged by error-tracking tools).
- Recommendation:
  - Avoid logging raw error responses in production.
  - Strip or hash sensitive fields before logging, or use a dedicated error-tracking service that sanitizes data.
  - Consider conditionally logging only in development (`if (import.meta.env.DEV)`).

---

### 5. Transport Security
- API calls use relative paths (proxied in dev) or `BASE_URL` from env.
- Risk: If `VITE_API_URL` is set to `http://` (non-HTTPS) in production, credentials and data will be transmitted in clear text.
- Recommendation:
  - Enforce HTTPS in all production environments.
  - Add a runtime check (e.g., in `getBaseUrl()` or `apiFetch`) that warns if the URL is not HTTPS in non-development modes.

---

### 6. Missing Security Headers (Server-Side Concern)
- The frontend cannot set security headers (CSP, HSTS, X-Frame-Options, etc.), but they are critical for production.
- Recommendation: Ensure the backend or reverse proxy (NGINX, Cloudflare, etc.) sends:
  - `Content-Security-Policy` (to mitigate XSS/data injection).
  - `Strict-Transport-Security` (HSTS) to enforce HTTPS.
  - `X-Content-Type-Options: nosniff`
  - `Referrer-Policy: strict-origin-when-cross-origin`
  - Permissions Policy to restrict unused features.

---

### 7. Dependency Management Hygiene
- Risk: Outdated or vulnerable dependencies can accumulate over time.
- Recommendation:
  - Integrate `npm audit` into CI/CD pipeline (block merges on new vulnerabilities).
  - Use tools like `Dependabot` or `Renovate` for automated PRs on dependency updates.
  - Periodically run `npm outdated` and review updates.

---

### Summary of Immediate Actions Before Production
1. Fix the vulnerability:
  ```bash
 npm audit fix --force # then run full test suite
  ```
2. Review and set environment variables correctly for production (especially `VITE_API_URL` and `VITE_API_KEY`).
3. Assess token storage strategy and consider migrating to HttpOnly cookies.
4. Add backend logout call to the frontend logout flow.
5. Sanitize error logging to avoid leaking sensitive data in console.
6. Verify that all API calls in production use HTTPS.
7. Ensure server-side security headers are configured (backend/infra responsibility).

---

### Final Note
This report focuses on frontend-observable risks. A full security assessment should also

```

24/05/2026, 13:54

include ****backend API review**** (authentication, authorization, input validation, rate limiting, etc.) and ****infrastructure scanning****. Addressing the items above will significantly reduce the attack surface before pushing to production.

End of Report